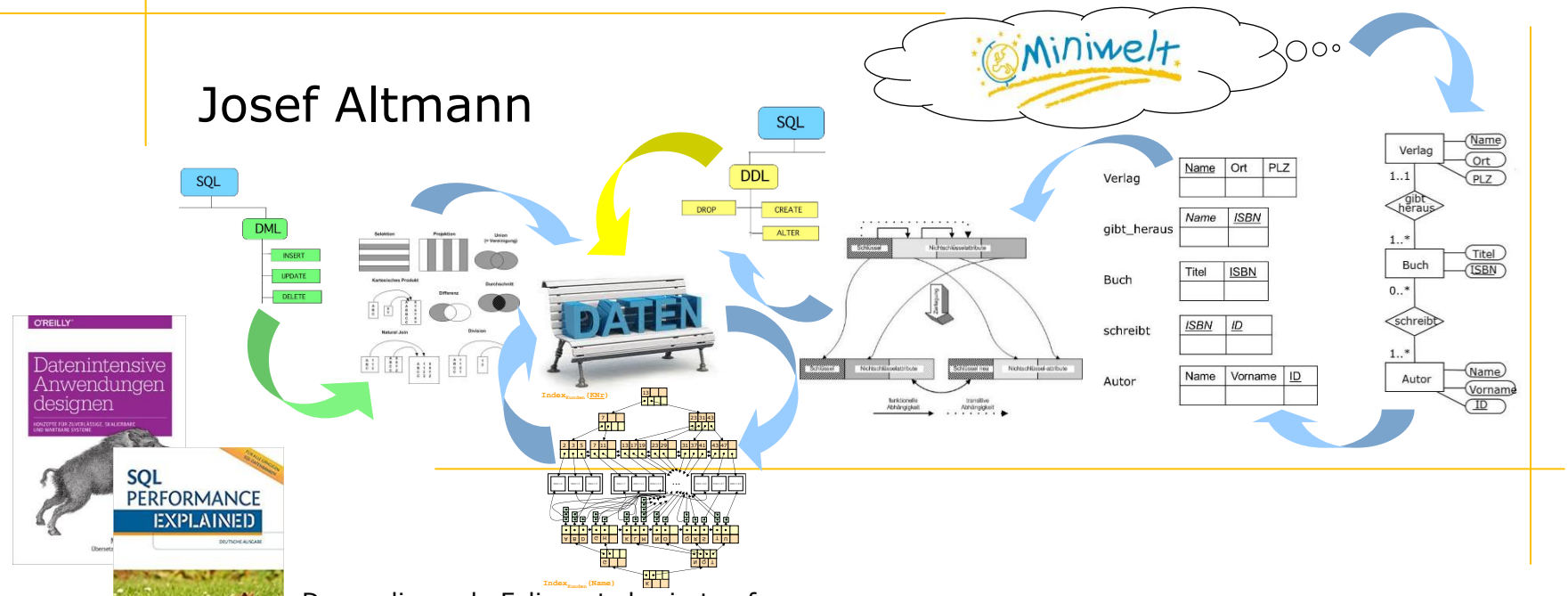
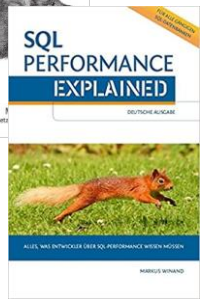
Modul 11

Physische Datenorganisation

Josef Altmann





Der vorliegende Foliensatz basiert auf:

Winand, M.: SQL Performance Explained, ISBN: 978-3-9503078-1-8, www.winand.at (2012)

Winand, M.: Use the Index, Luke! Anleitung zur SQL-Performance für Entwickler

Kleppmann, M.: Datenintensive Anwendungen designen, O'Reilly, 1. Auflage (2019)

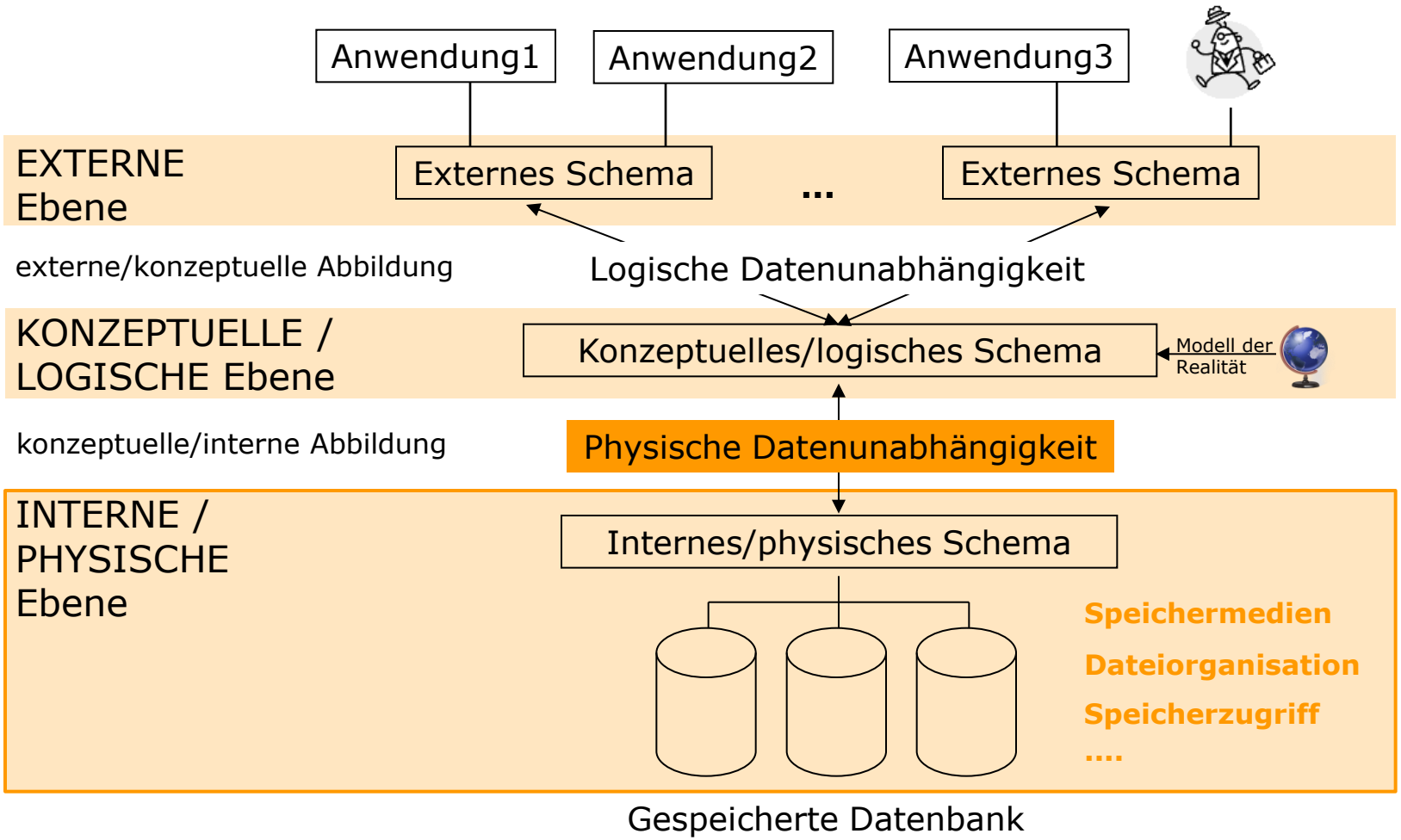
Überblick

- Internes Schema
- Abbildung konzeptuelle auf interne Ebene
- Dateiorganisation und Zugriffspfade
- Indexierung
- Indexstrukturen in SQL (ORACLE)

© 2026

Motivation

ANSI/X3/SPARC: Normungsvorschlag
für Systemarchitektur von DBMS



- Änderungen an der physischen Speicher- oder der Zugriffsstruktur (beispielsweise das Anlegen oder Entfernen einer Indexstruktur) hat keine Auswirkungen auf das logische/konzeptuelle Schema.

Speichermedien

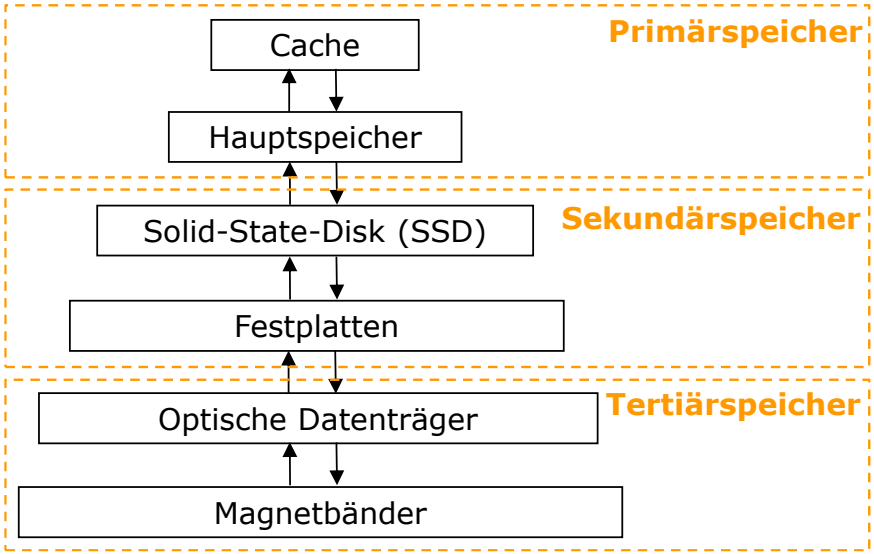
- Verschiedene Arten von Speichermedien sind für Datenbanksysteme relevant.
- Speichermedien lassen sich in Speicherhierarchie anordnen.

- Klassifizierung der Speichermedien nach:
  - Zugriffsgeschwindigkeit
  - Kosten pro Dateneinheit
  - Verlässlichkeit
    - Datenverlust durch Stromausfall oder Systemabsturz
    - Physische Fehler des Speichermediums
  - Flüchtige vs. persistente Speicher
    - Flüchtig (volatile): Inhalt geht nach Ausschalten verloren
    - Persistent (non-volatile): Inhalt bleibt auch nach Ausschalten

© 2026

Speicherhierarchie

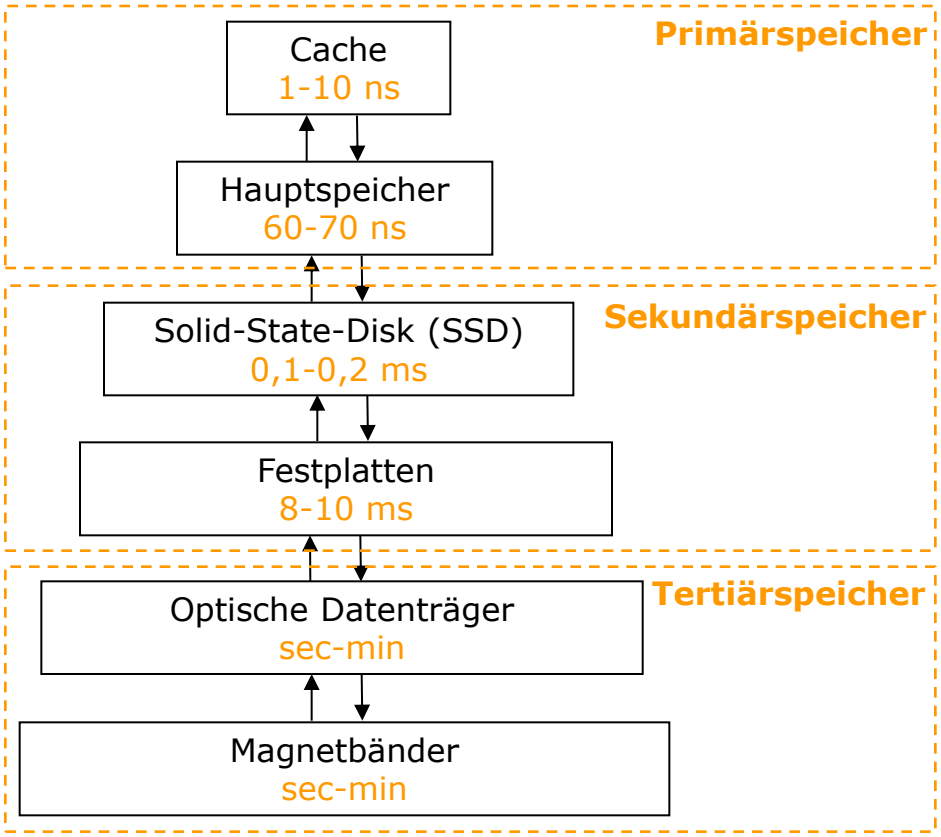
- Speichermedien können hierarchisch nach Geschwindigkeit und Kosten geordnet werden:
  - Primärspeicher: flüchtig, schnell, sehr teuer
    - Cache, Hauptspeicher
  - Sekundärspeicher: persistent, langsamer, günstiger
    - Magnetplatten, Flash Speicher
    - auch Online-Speicher genannt
  - Tertiärspeicher: persistent, sehr langsam, sehr günstig
    - Magnetbänder, optischer Speicher
    - auch Offline-Speicher genannt



- Datenbank muss mit Speichermedien auf allen Ebenen umgehen

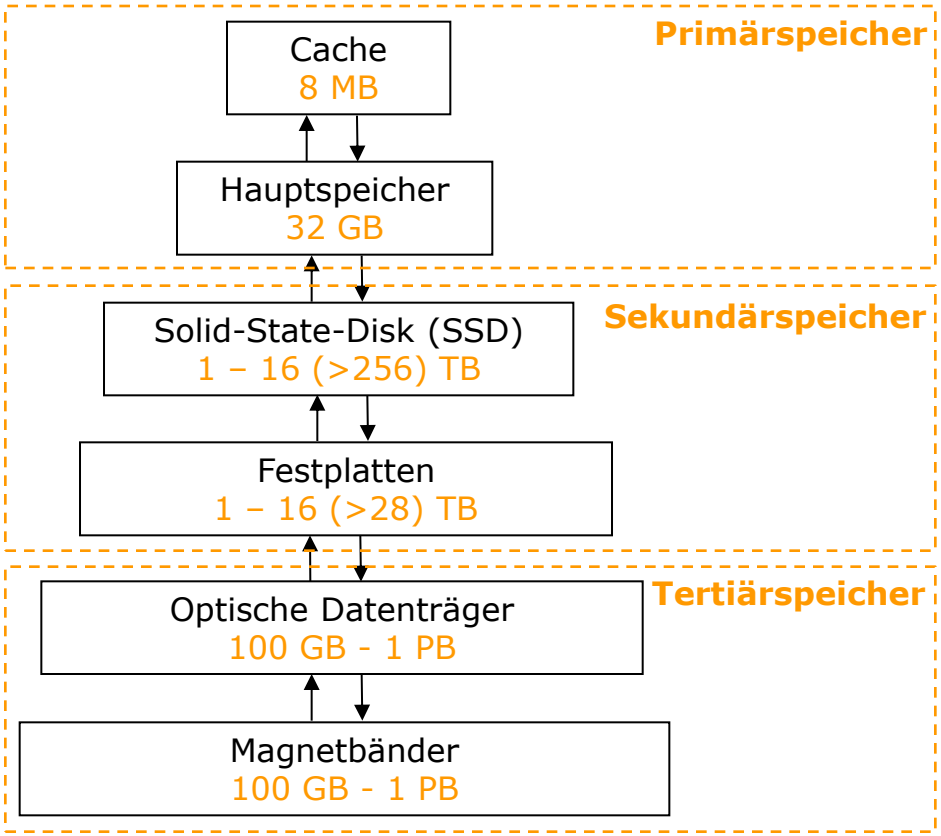
Speicherhierarchie
Zugriffszeiten

- Speichermedien nach Zugriffsgeschwindigkeit geordnet:



Speicherhierarchie
Kapazitäten

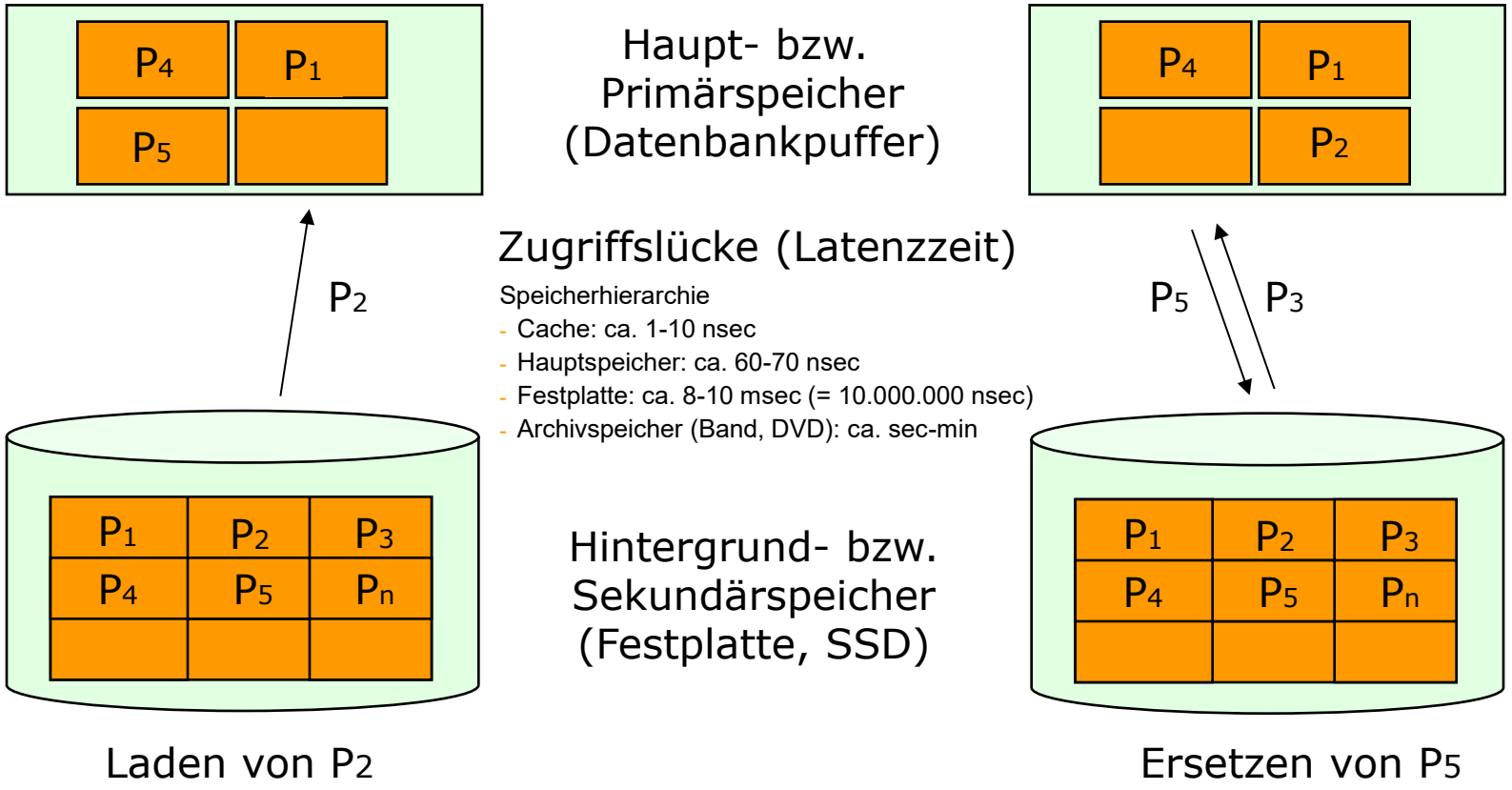
- Speichermedien nach Speicherkapazität geordnet:



© 2026

Speicherzugriff

- **Puffer**: Hauptspeicherbereich für Kopien von Platten-Blöcken
- **Puffer Manager**: Subsystem zur Verwaltung des Puffers
  - Anzahl der Plattenzugriffe soll minimiert werden
  - ähnlich der virtuellen Speicherverwaltung in Betriebssystemen



Dateiorganisation

- ... beschreibt die systemspezifische Realisierung der Datenbankobjekte (physische Speicherung)
  - **Datenstrukturen** (Aufbau der gespeicherten Datensätze)
  - **Zugriffsstrukturen** (Indexstrukturen, Hashing)
- ... bestimmt maßgeblich das Leistungsverhalten des Datenbanksystems

➢ Anwendungen sind von Änderungen des internen Schemas nicht betroffen (= physische Datenunabhängigkeit)

Abbildung konzeptuelle/logische Ebene auf interne Ebene

Konzeptuelle Ebene		Interne Ebene		Dateisystem/Platte
Relationen	→	Logische Dateien	→	Physische Dateien
Tupel	→	Datensätze (Records)	→	Seiten/Blöcke
Attributwerte	→	Felder	→	Bytes

- Abbildungen in beiden Richtungen sind durch Metadaten beschrieben, die im **Data Dictionary** verwaltet werden.

Exkurs: Data Dictionary

- Datenbankverzeichnis speichert Metadaten
  - Informationen über Relationen
    - Name der Relation
    - Name und Typen der Attribute jeder Relation
    - Name und Definition von Views
    - Integritätsbedingungen (z.B. Schlüssel und Fremdschlüssel)
  - Benutzerverwaltung
  - Statistische Beschreibung der Instanz
    - Anzahl der Tupel in der Relation/Speicherallokation
    - häufigste Werte
  - Physische Dateiorganisation
    - wie ist eine Relation gespeichert (sequentiell/Hash/...)
    - physischer Speicherort (z.B. Festplatte)
    - Dateiname oder Adresse des ersten Blocks auf der Festplatte
  - Information über Indexstrukturen
- Oracle stellt mehr als 2.300 Relationen und Sichten bereit

© 2026

Abbildung konzeptuelle/logische Ebene auf interne Ebene

Konzeptuelle Ebene		Interne Ebene		Dateisystem/Platte
Relationen	→	Logische Dateien	→	Physische Dateien
Tupel	→	Datensätze (Records)	→	Seiten/Blöcke
Attributwerte	→	Felder	→	Bytes

- Abbildungen in beiden Richtungen sind durch Metadaten beschrieben, die im **Data Dictionary** verwaltet werden.

- Abbildung auf physische Dateien:
  - Cooked files: DBMS verwendet Dateisystem des Betriebsystems (einfacher, code reuse)
  - Raw files: DBMS verwaltet Plattenbereich selbst (unabhängig von Betriebssystem, bessere Performance, mehr Kenntnisse erforderlich)

© 2026

Dateiorganisation und Zugriffsstrukturen

- Um Operationen (Anfragen, Hinzufügen, Löschen, Ändern) effizient durchführen zu können, setzt die interne Ebene des Datenbanksystems geeignete **Dateiorganisation** und **Zugriffspfade/strukturen** ein.

- Aufgaben
  - **Zuordnung eines Suchschlüssels** zu denjenigen physischen Datensätzen, die diese Wertekombination besitzen, d.h. Zuordnung zu der oder den Seiten der Datei, in denen diese Datensätze gespeichert sind
  - **Organisation der Seiten** unter dynamischen Bedingungen. Überlauf einer Seite → Aufteilen der Seite auf zwei Seiten

© 2026

Dateiorganisation

Anordnung der Datensätze in Dateien

- Speicherungsform der Datensätze in der Datendatei bestimmt die Effizienz der Zugriffs- und Modifkations-operationen auf den Datensätzen.
  - unsortierte Speicherung: Heap-Organisation
  - sortierte Speicherung: sequenzielle Organisation
  - gestreute Speicherung: Hash-Organisation

➢ üblich: Heap-Organisation

Zugriffsstrukturen

- Zugriffspfad beschleunigt Datenzugriff in der Datendatei, bspw. durch Index bzw. Indexdatei
  - Indexdatei hat Einträge der Form (↓Suchschlüssel, ↑Zeiger):
    - ↓Suchschlüssel: Attribut(menge) nach dem Daten gesucht werden
    - ↑Zeiger: Verweis auf Datensatz (TID - Tuple Identifier)
  - Suchschlüssel dürfen doppelt vorkommen (im Gegensatz zu Primärschlüssel)
  - Wichtigste Zugriffsarten sind
    - Punktanfrage: bspw. Kunde mit `KNr=33`
    - Mehrpunktanfrage: bspw. Kunden mit `Ort=Linz`
    - Bereichsanfrage: bspw. Kunden, mit `Umsatz > 100.000`
- Indexdatei kann wiederum in einer bestimmten Dateiorganisationsform gespeichert sein

Abbildung Relationen auf Sekundärspeicher

Mögliche Datenorganisation mit Zugriffsstrukturen

Eine Relation

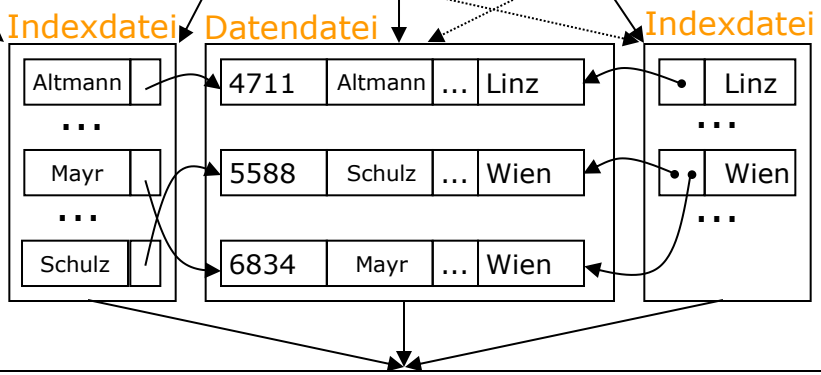
KNr	Name	...	Ort
4711	Altmann	...	Linz
5588	Schulz	...	Wien
6834	Mayr	...	Wien
...	...	...	

Zugriffsstrukturen
- B-Tree
- B+-Tree
- B*-Tree
- ...

Dateiorganisation:
- unsortierte Speicherung
- sortierte Speicherung
- gestreute Speicherung (Hash)
- ...

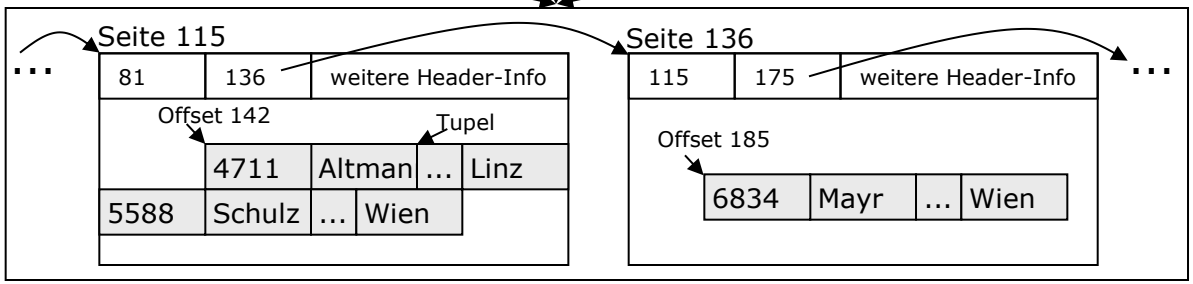
wird in mehrere **logische Dateien** abgebildet

(Datendatei und Indexdateien, die Zugriffsstrukturen enthalten)



und in einer **physischen Datei** gespeichert.

(Nutzdaten und Indexdaten werden meist physisch getrennt, bspw. separate Tablespaces in Oracle)

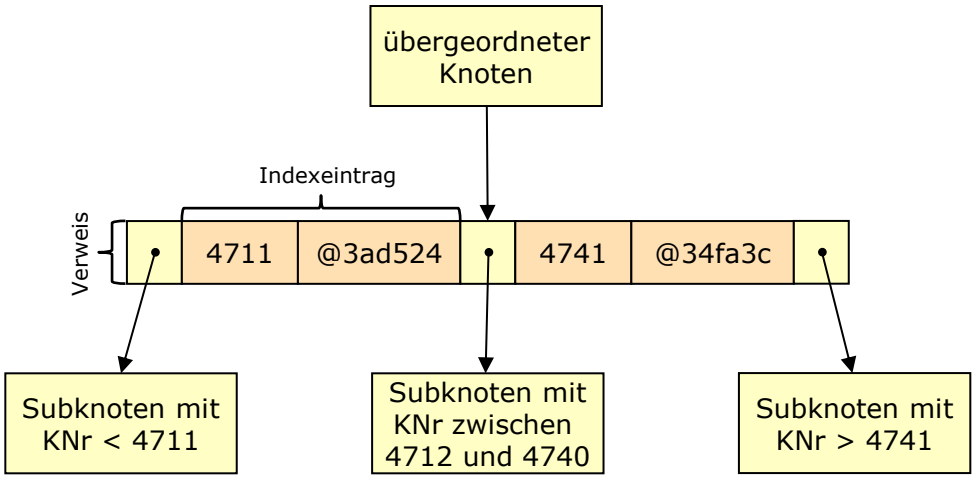


Blocken
Pufferverwaltung
Seitenersetzung
Fragmentierung
...
~
Betriebssystem (Cooked vs. Raw Devices)

Zugriffsstruktur der Indexdatei

B-Baum (B+-Baum, B*-Baum, etc.)

- Einfache Indexerstellung:
  - Datensatz mit **`KNr=4711`** befindet sich auf **`Seite 115`** ab **`Byte 142`**
- Was passiert, wenn Seite nicht mehr für alle Indexeinträge ausreicht?
  - Verwaltung mit B-Bäumen



B-Baum mit Ordnung m = 3

Ordnung **`m`** legt die Anzahl der Indexeinträge und Subknoten fest.

Für B-Baum mit Ordnung **`m`** gelten folgende Regeln:

- max. **`m-1`** Indexeinträge pro Knoten
- max. **`m`** Verweise auf Subknoten
- mind. **`m/2`** Verweise auf Subknoten
- **`k`** Verweise auf Subknoten erfordern **`k-1`** Indexeinträge
- B-Baum muss ausbalanciert sein [1]

- Für sämtliche Endknoten ist der Abstand zum Startknoten gleich
- B-Bäume sind typischerweise sehr breit und besitzen pro Knoten sehr viele Subknoten

[1] B-Baum-Simulator: www.cs.usfca.edu/~galles/visualization/BTree.html

Zugriffsstruktur der Indexdatei

B-Baum - Rechenbeispiel

- Annahmen:
  - Jeder Knoten entspricht einer Seite
  - Seitengröße beträgt 8 kByte
  - Pro Indexeintrag werden 16 Byte benötigt
    - je 8 Byte für eine 64-Bit-ID und für eine Adressangabe
  - Verweis zu einem anderen Knoten benötigt 4 Byte

  - Damit können max. 409 Indexeinträge sowie 410 Verweise pro Knoten/Seite gespeichert werden. Es handelt sich um einen B-Baum der Ordnung 410.
  - Annahme: Jeder Knoten ist zu ca. 60 % gefüllt
    - ergibt 245 Indexeinträge pro Knoten

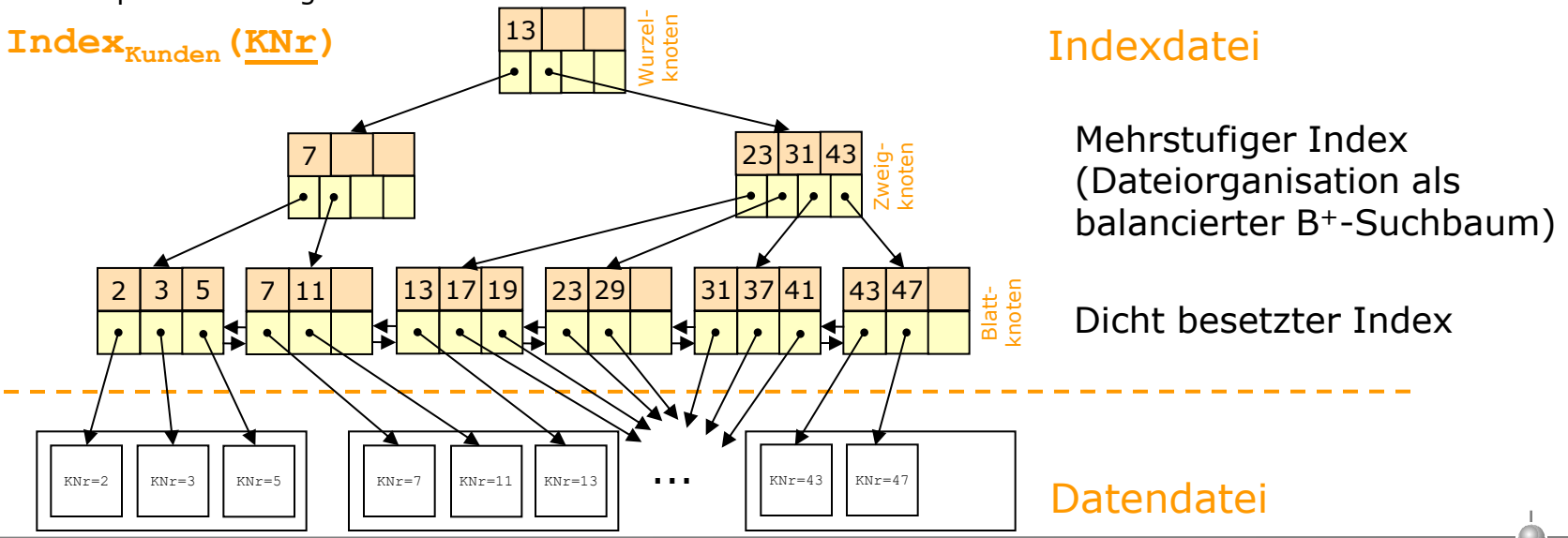
  - Mit einem Baum der Tiefe 3 können 245^3, also ca. 14 Millionen Datensätze indiziert werden.
  - Mit einem Baum der Tiefe 4 können 3,6 Milliarden Datensätze indiziert werden.

  - ➢ Trotz dieser enormen Anzahl reicht es aus, nur vier Knoten zu durchsuchen, um die Adresse des gesuchten Datensatzes zu ermitteln.

© 2026

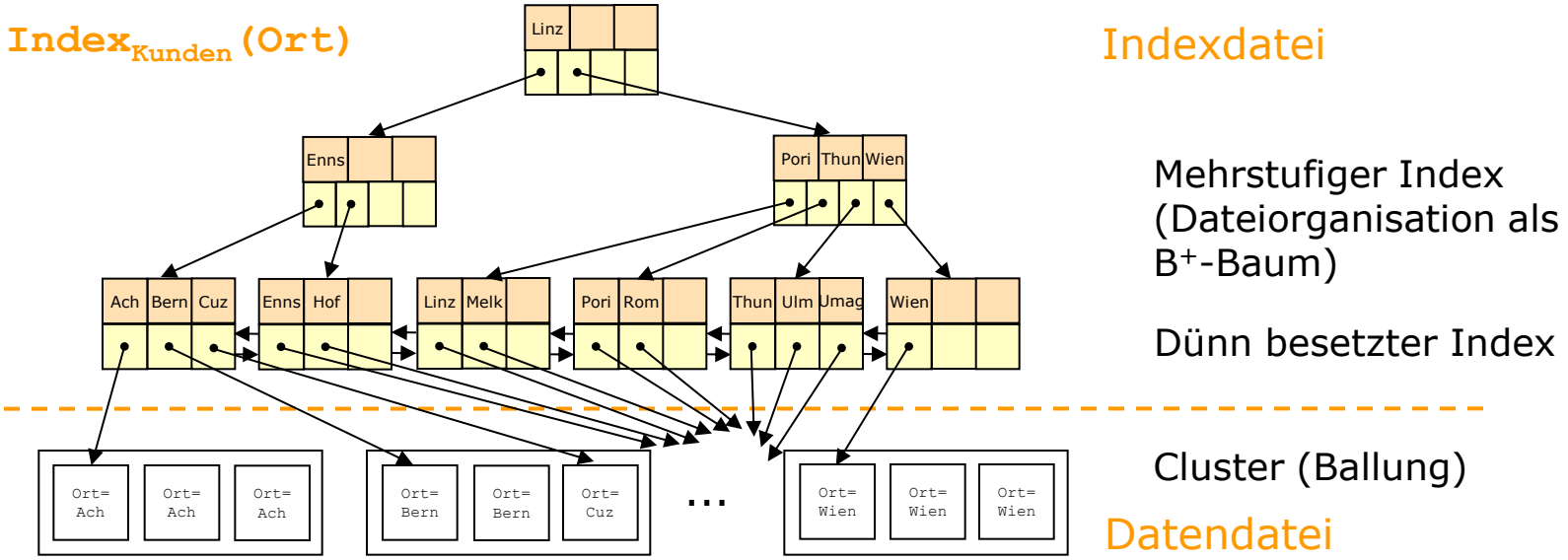
Indexstrukturen
Primärindex

- Datendatei
  - sequentiell nach Suchschlüssel geordnet
  - Suchschlüssel ist eindeutig
    - d.h., zu jedem Wert eines Suchschlüssls gibt es nur einen Datensatz in der Datendatei - trifft bei Primärschlüssel oder Schlüsselkandidaten zu
- Indexdatei
  - sequentiell nach Suchschlüssel geordnet
  - Primärindex garantiert, dass keine Duplikate enthalten sind
- Effiziente Zugriffsarten:
  - Punkt- und Bereichsanfragen
  - nicht-sequentieller Zugriff (random access)
  - sequentieller Zugriff nach Suchschlüssel sortiert



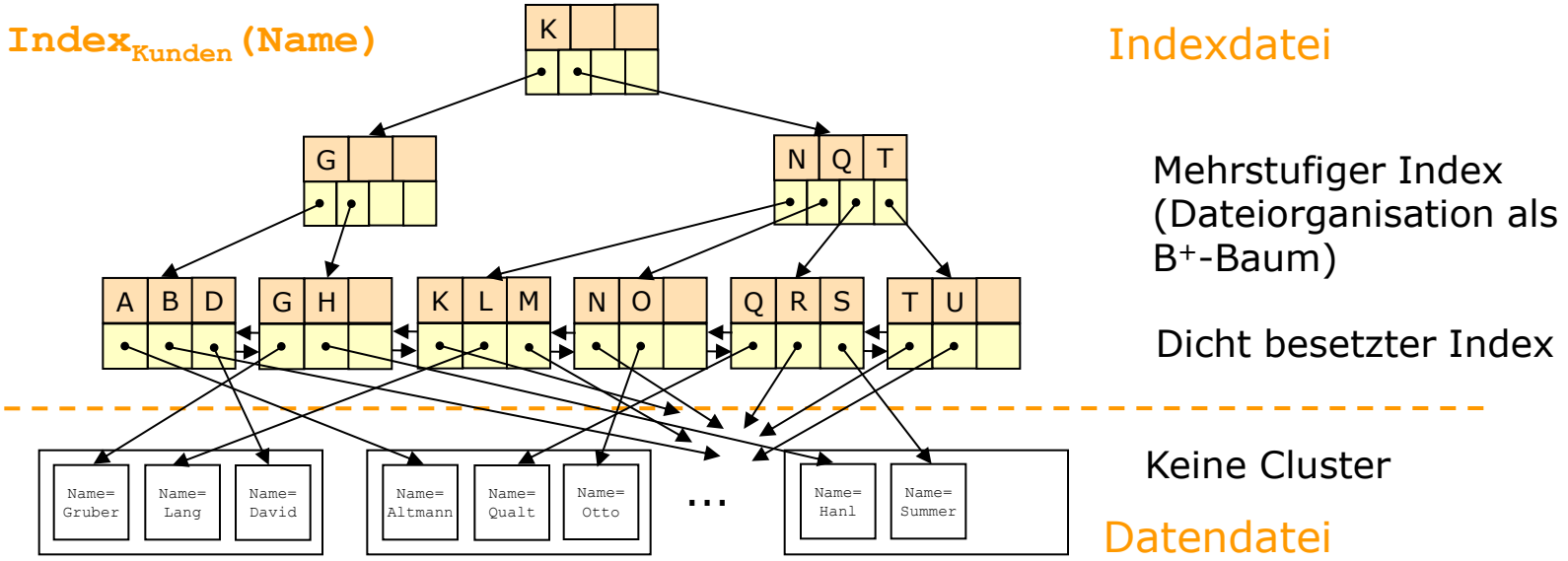
Indexstrukturen
Clusterindex

- Datendatei
  - sequentiell nach Suchschlüssel geordnet
  - Suchschlüssel kann mehrfach vorkommen
- Indexdatei
  - sequentiell nach Suchschlüssel geordnet
- Effiziente Zugriffsarten:
  - Punkt-, Mehrpunkt- und Bereichsanfragen
  - nicht-sequentieller Zugriff (random access)
  - sequentieller Zugriff nach Suchschlüssel sortiert



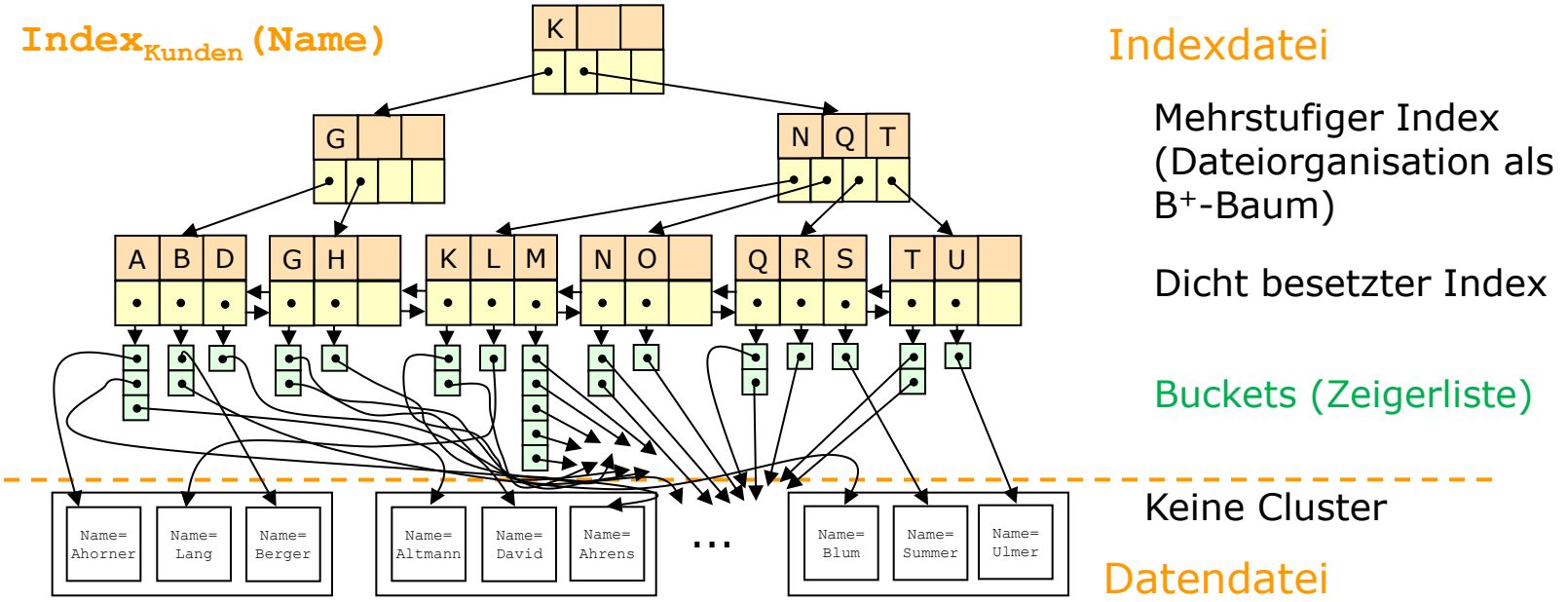
Indexstrukturen
Sekundärindex

- Datendatei
  - **nicht** nach Suchschlüssel geordnet
  - Suchschlüssel kann mehrfach vorkommen
- Indexdatei
  - sequentiell nach Suchschlüssel geordnet
- Effiziente Zugriffsarten:
  - sehr schnell für Punktanfragen
  - Mehrpunkt- und Bereichsanfragen: gut wenn nur kleiner Teil der Tabelle zurückgeliefert wird (wenige %)
  - besonders fur nicht-sequentiellen Zugriff (random access) geeignet



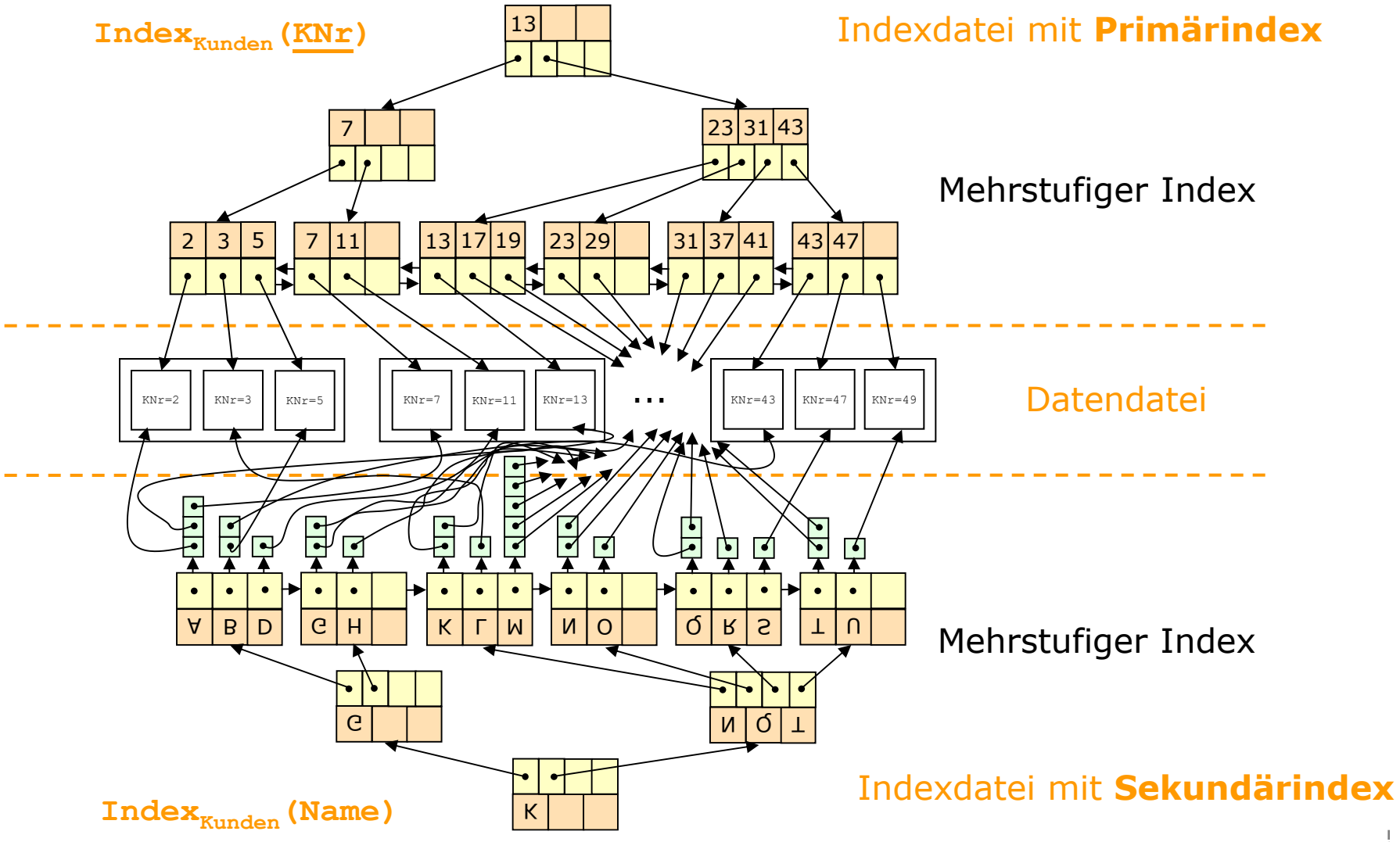
Indexstrukturen
Sekundärindex

- Zwei Möglichkeiten für Zeigerabbildung
  - Mehrfache Indexeinträge
    - ein Indexeintrag für jeden Datensatz
  - Buckets
    - ein Indexeintrag für jeden Suchschlüssel
    - Indexeintrag zeigt auf ein Bucket
    - Bucket enthält Zeiger auf Datensätze



Indexstrukturen

Kombination Primär- und Sekundärindex



Indexstrukturen

Primär- versus Sekundärindex

- nur ein Primärindex (bzw. ein Clustered Index) pro Tabelle möglich
- mehrere Sekundärindizes pro Tabelle möglich
- Sekundärindex für schnellen Zugriff auf alle Felder, die nicht Suchschlüssel des Primärindex sind

- Beispiel: Tabelle **`Kunde`** mit Primärindex auf **`KNr`**
  - Finde alle Kunden mit **`Ort=Linz`**
  - Finde alle Kunden mit **`Umsatz 100.000 bis 200.000`**
- Ohne Index können diese Anfragen nur durch sequentielles Lesen aller Knoten beantwortet werden – sehr langsam
- Sekundärindex für schnellen Zugriff erforderlich

© 2026

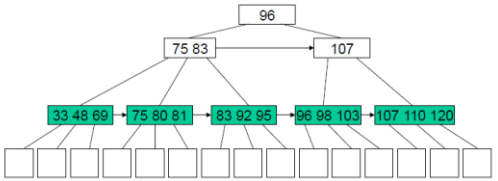
Indexstrukturen
Gegenüberstellung von Indextypen

- Alle Indextypen machen Punktanfragen erheblich schneller.
- Index erzeugt Kosten bei Datenaktualisierung (DML-Operationen) --> Index muss ebenfalls aktualisiert werden.
- Dicht/Dünn besetzter Index und Primär-/Sekundärindex:
  - Primärindex kann dicht oder dünn besetzt sein
  - Sekundärindex ist immer dicht besetzt
- Sortiert lesen (=sequentielles Lesen nach Suchschlüsselordnung):
  - mit Primärindex schnell
  - mit Sekundärindex teuer, da sich aufeinander folgende Datensätze auf unterschiedlichen Seiten/Blöcken befinden (können)
- Dicht versus dünn besetzter Index:
  - Dünn besetzter Index braucht weniger Platz
  - Dünn besetzter Index hat geringere Kosten beim Aktualisieren
  - Dicht besetzter Index erlaubt bestimmte Anfragen zu beantworten, ohne dass Datensätze gelesen werden müssen (Covering Index)

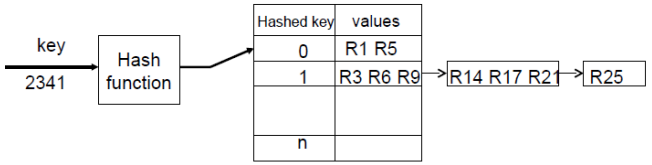
© 2026

Schlüsselvergleich vs. Schlüsseltransformation

- Baum-basiert: Schlüsselvergleich
  - Zuordnen von Schlüsselwerten zu Adressen (Zeigern) in Indexdateien
  - Indexsequenzielle Organisation auf Basis von B^+-Bäumen
  - Effizient für Bereichsanfragen: Suchen von Sätzen mit bestimmter Werteausprägung ('<', '>', `BETWEEN`, `ORDER BY`, `MIN/Max`, ...)



- Hash-basiert: Schlüsseltransformation
  - Berechnet Tupeladresse aufgrund Formel aus Schlüsselwerten (statt Indexeinträgen nur Berechnungsvorschrift gespeichert)
  - Hash-Verfahren: meist wählt Datenbank geeignete Hash-Funktion aus
  - Effizient für Punktanfragen: Suchen von Sätzen mit bestimmter Werteausprägung ('=', Gleichverbund)



Statische vs. dynamische Zugriffstruktur

- **Statische Zugriffsstruktur**: optimal nur bei bestimmter (fester) Anzahl von verwaltenden Datensätzen
- **Dynamische Zugriffsstruktur**: unabhängig von der Anzahl der Datensätze optimal
  - dynamische Adresstransformationsverfahren verändern dynamisch Bildbereich der Transformation
  - dynamische Indexverfahren verändern dynamisch Anzahl der Indexstufen ⇨ in DBS üblich

Überblick

- Internes Schema
- Abbildung konzeptuelle auf interne Ebene
- Dateiorganisation und Zugriffspfade
- Indexierung
- Indexstrukturen in SQL (ORACLE)

© 2026

Indexstrukturen in SQL

- SQL-Standard definiert keine Syntax für Indizes
  - es hat sich aber eine gebräuchliche Syntax durchgesetzt
  - diese wird anhand von Oracle dargestellt

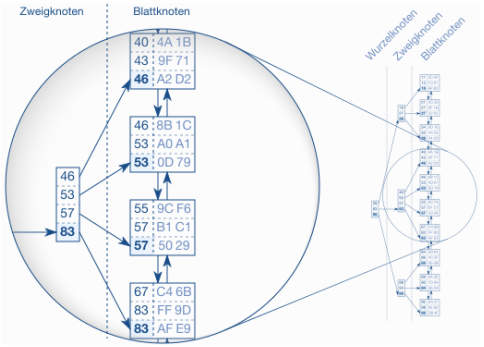
- Datenbanksysteme realisieren spezifische Indizes:
  - Indizes werden als eigenständige Schemaobjekte behandelt.
  - Indizes-Verwaltungsinformationen werden von Datenbanksystemen automatisch erstellt.
  - Indizes sind in der Regel als B^+-Bäume realisiert

- Vorteile:
  - Beschleunigung des Zugriffs
  - Gewährleistung der Cluster-eigenschaft für Tabellen

- Nachteile:
  - erhöhter Verwaltungsaufwand
  - höherer Speicheraufwand (Indizes benötigen Speicherplatz)

© 2026

Indizes 1/20
ORACLE



- Standardindex ist als B^+-Baum aufgebaut.
- Index kann max. 32 Spalten umfassen.
- Tabellen sind uneingeschränkt mehrfach indizierbar.
- Funktionsbasierte Indizes sind auf den Ergebniswerten von Funktionsaufrufen über Spalten definiert.
- Zusammengesetzte Indizes umfassen mehrere Spalten.
- Bitmap-Indizes speichern Bitmatrizen für Aufzählungsattribute.
- Covering Indizes vermeiden Tabellenzugriffe.
- Cluster-Indizes erlauben Clusterung auch mehrerer Tabellen.
- Speicherorganisation erfolgt als Heap- oder Index-organisierte Tabelle.
- ...

Indizes 2/20
ORACLE - Eindeutige Indizes (Primärindex)

- Indizes werden automatisch für
  - `PRIMARY KEY`- und
  - `UNIQUE`-Constraints

  angelegt.

`CREATE INDEX`-Anweisung

```
CREATE [UNIQUE] INDEX Indexname
ON Tabelle (Attribut1[ASC, DESC], ..., Attributn[ASC, DESC]);
```

Implizite `CREATE UNIQUE INDEX`-Anweisung

```
CREATE TABLE Tabelle
  (Attribut1  PRIMARY KEY USING INDEX
              (CREATE UNIQUE INDEX Indexname ON
              Tabelle(Attribut1)),
   Attributn);
```

Indizes 3/20
ORACLE - Eindeutige Indizes (Primärindex)

CREATE TABLE-Anweisung

```
CREATE TABLE Employee (
  employee_id                NUMBER           NOT NULL,
  first_name                 VARCHAR2(1000)   NOT NULL,
  last_name                  VARCHAR2(1000)   NOT NULL,
  date_of_birth              DATE             NOT NULL,
  phone_number               VARCHAR2(1000)   NOT NULL,
  CONSTRAINT employee_pk PRIMARY KEY (employee_id)
  );
```

```
SELECT *
FROM Employee
WHERE employee_id = 123;
```



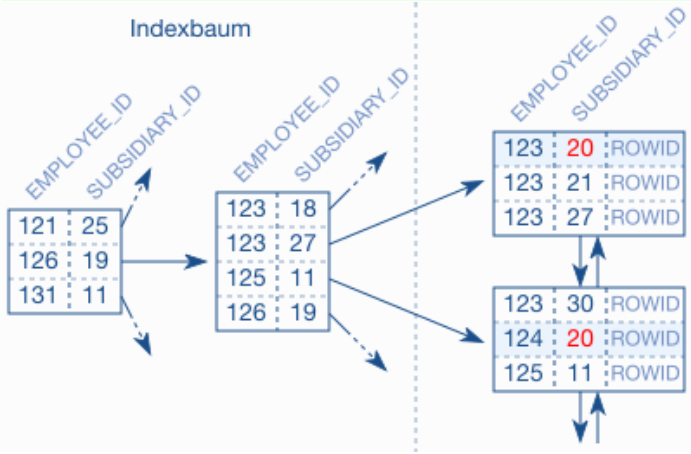
Indizes 4/20
ORACLE - Zusammengesetzter Primärschlüssel

- Index umfasst alle Spalten des Primärschlüssels
- Spaltenreihenfolge im Index ausschlaggebend!

CREATE TABLE-Anweisung

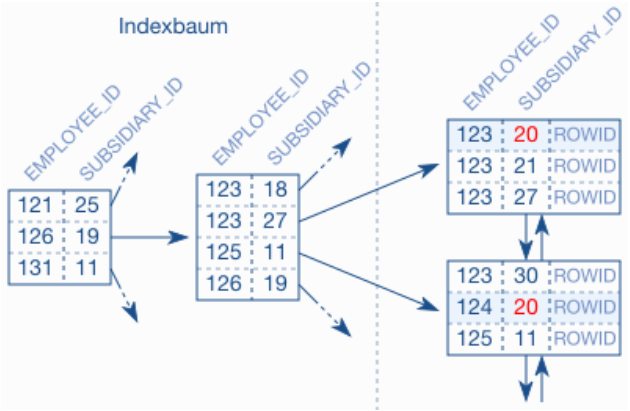
```
CREATE TABLE Employee (
  employee_id NUMBER NOT NULL,
  subsidiary_id NUMBER NOT NULL,
  ...
  CONSTRAINT employee_pk PRIMARY KEY (employee_id, subsidiary_id));

// Index wie folgt automatisch definiert
CREATE UNIQUE INDEX employee_pk_index ON employee (employee_id, subsidiary_id);
```



Indizes 5/20

ORACLE - Zusammengesetzter Primärschlüssel




```
SELECT *
FROM Employee
WHERE employee_id = 123
AND subsidiary_id = 30;
```

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT			1
TABLE ACCESS (BY INDEX ROWID)	EMPLOYEES	1	1
INDEX (UNIQUE SCAN)	EMPLOYEES_PK	1	1
Access Predicates			
AND			
EMPLOYEE_ID=123			
SUBSIDIARY_ID=30			

Index wird verwendet!

```
SELECT *
FROM Employee
WHERE subsidiary_id = 20;
```

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT			2
TABLE ACCESS (FULL)	EMPLOYEES	1	2
Filter Predicates			
SUBSIDIARY_ID=20			

Index wird nicht verwendet!

```
CREATE UNIQUE INDEX employee_pk ON employee (subsidiary_id, employee_id);
```

Indizes 6/20
ORACLE - Sekundärindizes

```
SELECT first_name, last_name, subsidiary_id, phone_number
FROM Employee
WHERE last_name = 'WINAND'
AND subsidiary_id = 30;
```

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |   30 |
|*1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |   30 |
|*2 |  INDEX RANGE SCAN           | EMPLOYEES_PK |   40 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  1 - filter("LAST_NAME"='WINAND')
  2 - access("SUBSIDIARY_ID"=30)
```

Hoher Cost-Wert, obwohl Empoyees_PK-Index verwendet wird!

- Aufführungsplan zeigt
  - `INDEX RANGE SCAN` liefert ersten Zweigknoten mit `SUBSIDIARY=30`
  - Blattknoten liefert Zeigerliste mit `ROWIDs`, die Bedingung `SUBSIDIARY=30` erfüllen
  - `TABLE ACCESS BY INDEX ROWID` liefert alle Zeilen mit allen Spalten
  - Wenn `LAST_NAME` verfügbar, erst dann kann Rest von `WHERE`-Klausel ausgewertet werden (`LAST_NAME = 'WINAND'`)

© 2026

Indizes 7/20
ORACLE - Sekundärindizes

```
CREATE INDEX emp_name ON employee (last_name);
```

```
-----------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    3 |
|* 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    3 |
|* 2 |   INDEX RANGE SCAN           | EMP_NAME  |    1 |    1 |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("SUBSIDIARY_ID"=30)
   2 - access("LAST_NAME"='WINAND')
```

Niedriger Cost-Wert, da Index nur wenige Zeile liefert!

- Aufführungsplan zeigt
  - `INDEX RANGE SCAN` liefert nur eine Zeile (wenn nur ein Mitarbeiter mit `LAST_NAME = 'WINAND'` vorhanden ist)
  - `TABLE ACCESS BY INDEX ROWID` muss daher nur eine Zeile aus der Tabelle laden

© 2026

Indizes 8/20
ORACLE - Funktionsbasierte Indizes

- Wird in der `WHERE`-Klausel ein Ausdruck oder eine Funktion verwendet, so kann dabei kein Standardindex genutzt werden.

```
SELECT first_name, last_name, phone_number
FROM Employee
WHERE UPPER(last_name) = UPPER('winand');
```

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |   10 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |   10 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(UPPER("LAST_NAME")='WINAND')
```

`TABLE ACCESS FULL`, obwohl ein Index auf `LAST_NAME` existiert!

`UPPER`-Funktion wird als `BLACKBOX()` betrachtet!

Funktionsbasierter Index

```
CREATE INDEX emp_up_name ON Employee (UPPER(last_name));
```

Achtung: `UPPER` und `LOWER` werden von ORM-Tools manchmal ohne Wissen der Entwickler verwendet. Hibernate verwendet zum Beispiel die `LOWER`-Funktion, um Case-insensitive Suchen umzusetzen.

© 2026

Indizes 9/20
ORACLE - Zusammengesetzte Indizes

- Kandidaten: **Spaltenkombinationen**, die vorwiegend gemeinsam in `WHERE`-Klauseln vorkommen (`AND`-Verknüpfung)
- Spalten, die auch in Einzelsuchkriterien verwendet werden, sollten als **führende Komponenten** des Index definiert werden (Indiziere zuerst für die Ist-gleich-Bedingungen, dann für andere)

  Index auf Spalten `x`, `y`, `z` der Tabelle `t` bei häufigen Abfragen auf die Kombination (`y`, `z`):

```
CREATE INDEX comp_index ON t(y,z,x);
```

Indizes 10/20

ORACLE - Zusammengesetzte Indizes

CREATE TABLE/INDEX-Anweisung

```
CREATE TABLE Employee (
  employee_id      NUMBER              NOT NULL,
  first_name       VARCHAR2(1000)      NOT NULL,
  last_name        VARCHAR2(1000)      NOT NULL,
  date_of_birth    DATE                NOT NULL
    CONSTRAINT employee_pk PRIMARY KEY (employee_id)
  );
CREATE INDEX emp_comp_index ON Employee (last_name, first_name, date_of_birth);
```

emp_comp_index wird verwendet

```
SELECT * FROM Employee WHERE  last_name = 'WINAND';

SELECT * FROM Employee WHERE  last_name = 'WINAND' AND first_name = 'JOE';

SELECT * FROM Employee WHERE  last_name = 'WINAND'
                              AND first_name = 'JOE'
                              AND date_of_birth = '1966-08-09';
```

emp_comp_index wird nicht verwendet

```
SELECT * FROM Employee WHERE  first_name = 'JOE';

SELECT * FROM Employee WHERE  date_of_birth = '1966-08-09';

SELECT * FROM Employee WHERE  last_name = 'WINAND' -- emp_comp_index für last_name
                              AND date_of_birth = '1966-08-09'; -- scan notwendig
```

© 2026

Indizes 11/20
ORACLE - Bitmap-Index

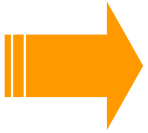
- Index auf Spalten mit wenig verschiedenen Wertausprägungen
- 2-dimensionale Matrix mit booleschen Werten
  - 1.Dimension bildet die `ROWID`-Werte der Tupel
  - 2.Dimension enthält Werte des indizierten Attributs
- Vorteile:
  - effiziente Auswertung von logischen Verknüpfungen (wie `AND`, `OR`, `NOT`)
  - effiziente Auswertung von `COUNT()` (kein Zugriff auf Basistabelle notwendig)
- Einsatzgebiet: große Datenbanken bspw. Data-Warehouse

EMPLOYEE

ROWID	Name	City	Gender	State
1	King	San Francisco	Male	CA
2	Kochar	San Jose	Male	NV
3	De Haan	Tracy	Female	CA
4	Hunold	Pleasanton	Male	NY
5	Rajs	Lake Tahoe	Divers	NV
6	Davies	Las Vegas	Unknown	CA
7	Matos	New York	Male	NY
8	Abel	New York	Female	CA

```
CREATE BITMAP INDEX emp_gender
ON Employee(gender);
```

```
CREATE BITMAP INDEX emp_state
ON Employee(state);
```



BITMAP-INDEX EMP_STATE

ROWID	State='CA'	State='NV'	State='NY'	...
1	1	0	0	
2	0	1	0	
3	1	0	0	
4	0	0	1	
5	0	1	0	
6	1	0	0	
7	0	0	1	
8	1	0	0	

BITMAP-INDEX EMP_GENDER

ROWID	Gender='Male'	Gender='Female'	Gender='Divers'	...
1	1	0	0	
2	1	0	0	
3	0	1	0	
4	1	0	0	
5	0	0	1	
6	0	0	0	
7	1	0	0	
8	0	1	0	

Indizes 12/20
ORACLE - Covering Index (Index-Only-Scan)

CREATE TABLE-Anweisung

```
CREATE TABLE Sales (
  sale_id         NUMBER        NOT NULL,
  employee_id     NUMBER        NOT NULL,
  subsidiary_id   NUMBER        NOT NULL,
  sale_date       DATE          NOT NULL,
  eur_value       NUMBER(17,2) NOT NULL,
  product_id      NUMBER        NOT NULL,
  quantity        NUMBER        NOT NULL,
  CONSTRAINT sales_pk
    PRIMARY KEY (sale_id),
  CONSTRAINT sales_emp_fk
    FOREIGN KEY(subsidiary_id, employee_id)
    REFERENCES employees(subsidiary_id, employee_id)
);
```

```
CREATE INDEX sales_sub_eur
ON Sales(subsidiary_id, eur_value);
```

Indizes 13/20
ORACLE - Covering Index (Index-Only-Scan)

- Vermeidung des Tabellenzugriffes, wenn Abfrage die gesuchten Spalten aus dem Index entnehmen kann.
- Aggregierende Abfragen sind gute Kandidaten für Covering Indizes.

```
SELECT SUM(eur_value)
FROM Sales
WHERE subsidiary_id = 30;
```

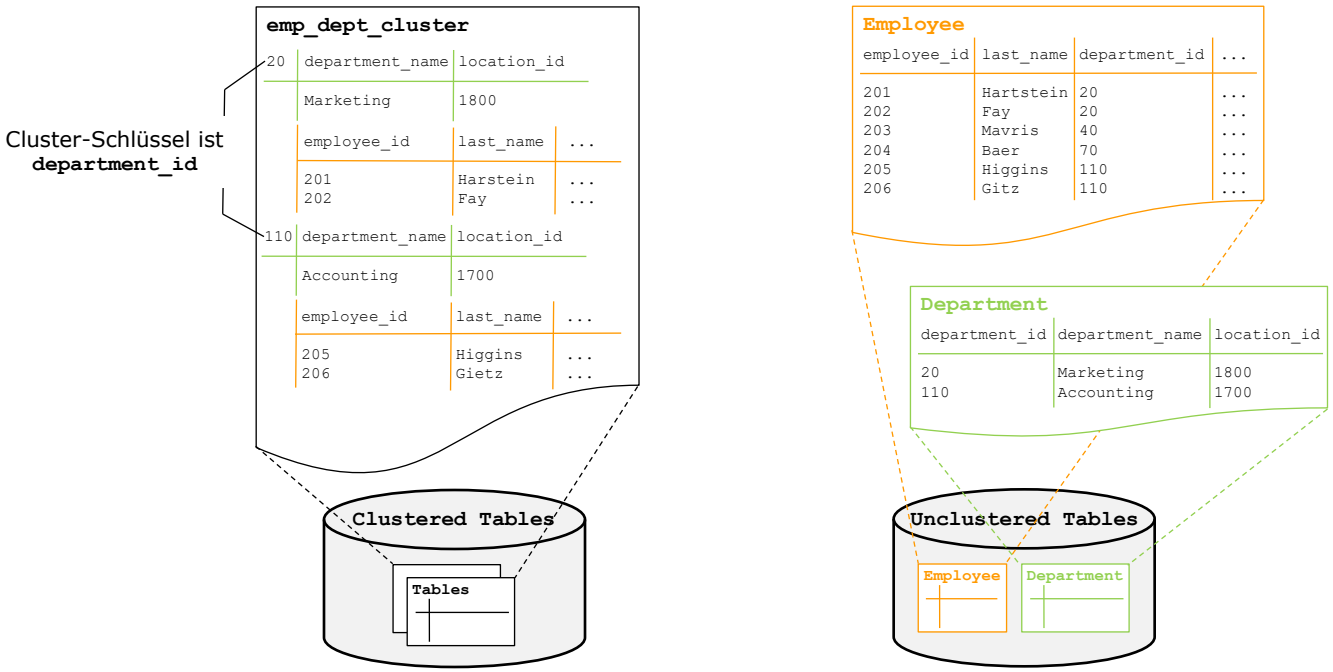
```
-------------------------------------------------------------
| Id  | Operation         | Name          |  Rows | Cost |
-------------------------------------------------------------
|   0 | SELECT STATEMENT  |               |     1 |  104 |
|   1 |  SORT AGGREGATE   |               |     1 |      |
|*  2 |   INDEX RANGE SCAN| SALES_SUB_EUR | 40388 |  104 |
-------------------------------------------------------------
```

Es erfolgt kein Tabellenzugriff (TABLE ACCESS BY ROWID)!

Aufsummierung von ca. 40.000 Zeilen - Ersparnis von 40.000 Tabellenzugriffen.

Indizes 14/20
ORACLE - Cluster-Indizes

- Cluster speichert logisch zusammengehörende Datensätze unterschiedlicher Tabellen in einem gemeinsamen physischen Speicherort (bspw. in einem Block).
- Cluster-Schlüssel umfasst alle Attribute der Tabellen, die über die gleiche Attributausprägung verfügen.
- Cluster-Index indiziert den Cluster-Schlüssel.



Indizes 15/20
ORACLE - Cluster-Indizes

CREATE CLUSTER-Anweisung

```
CREATE CLUSTER emp_dept_cluster
    (department_id NUMBER(4));
```

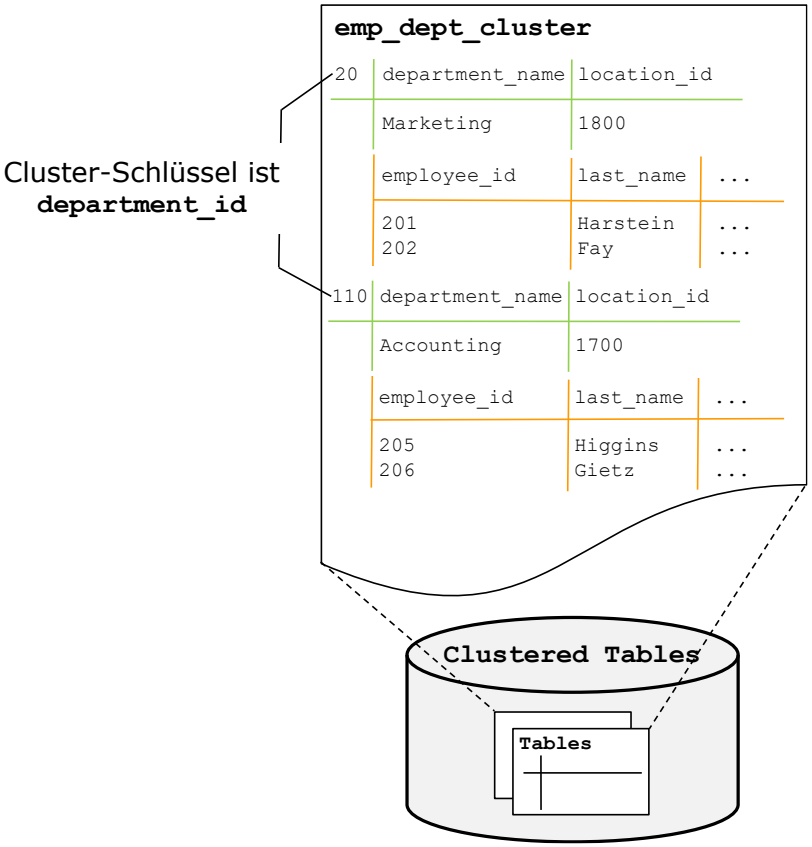
CREATE CLUSTER INDEX-Anweisung

```
CREATE INDEX idx_emp_dept_cluster
    ON CLUSTER emp_dept_cluster;
```

Tabellen im Cluster speichern

```
CREATE TABLE Department (
  department_id NUMBER(4) PRIMARY KEY,
  department_name VARCHAR2(20) NOT NULL)
CLUSTER emp_dept_cluster (department_id);

CREATE TABLE Employee (
  employee_id NUMBER(5) PRIMARY KEY,
  employee_name VARCHAR2(20) NOT NULL,
  department_id NUMBER(4))
CLUSTER emp_dept_cluster (department_id);
```



Indizes 16/20
ORACLE - Cluster-Indizes

- Vorteile:
  - Reduktion von I/O-Zugriffen und Zugriffszeiten bei Verbundoperationen über den Cluster-Schlüssel
  - Speicherreduktion, da Cluster-Schlüssel nur einmal gespeichert ist.

- Nachteile:
  - Zugriffszeiten ohne Cluster-Schlüssel im Suchprädikat können erheblich ansteigen, da Datenergebnismenge auf vielen Blöcken verteilt sein kann.
  - Änderungsoperationen (DML) sind zeitaufwändig (Reorganisation der Speicherstrukturen)

Indizes 17/20
Speicherorganisation

- Heap- versus Index-organisierte Tabellen
  - Tabellen, die nur einen Index haben, werden am besten als Index-organisierte Tabelle umgesetzt.
  - Tabellen mit mehreren Indizes können von der Heap-Struktur profitieren.
    - Dennoch kann man die Indizes jeweils für einen Index-Only-Scan auslegen, sodass der Zugriff auf die Heap-Struktur entfällt.
    - Damit erhält man die `SELECT`-Performance einer Index-organisierten Tabelle, ohne die Zugriffe über andere Indizes zu verlangsamen.

- Unterstützung von Index-organisierten Tabellen ist von Datenbank zu Datenbank sehr unterschiedlich!

© 2026

Indizes 18/20
ORACLE - Heap-Tabelle

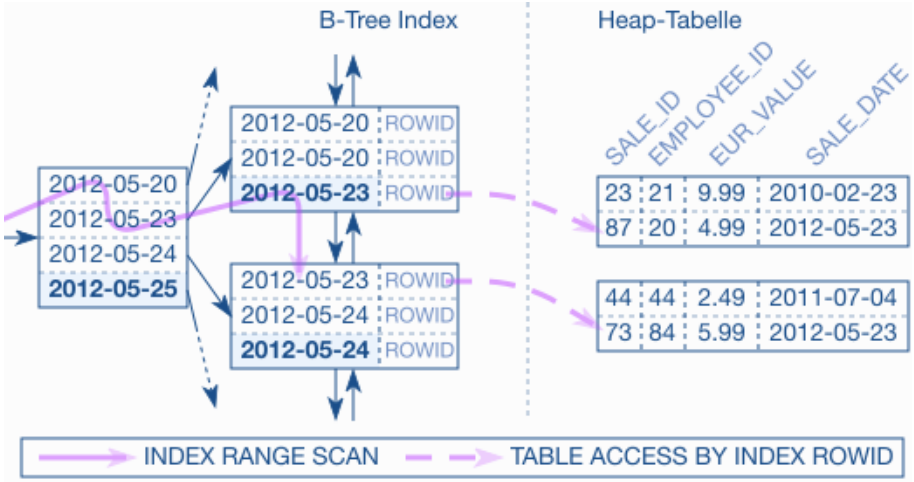
- Tabellen werden **standardmäßig als Heap-Tabellen** gespeichert.
- Zeilen werden in ungeordneter Reihenfolge abgelegt.

CREATE TABLE-Anweisung als Heap-Tabelle

```
CREATE TABLE Sales (
  sale_id           NUMBER          NOT NULL,
  employee_id       NUMBER          NOT NULL,
  eur_value         NUMBER(17,2)    NOT NULL,
  sale_date         DATE            NOT NULL,
  CONSTRAINT sales_pk PRIMARY KEY (sale_id)
);

CREATE INDEX sales_date
ON Sales (sale_date);

SELECT sale_id
FROM Sales
WHERE sale_date =
      to_date('2012-05-23', 'yyyy-mm-dd');
```



OPERATION	OBJECT_NAME	OPTIONS
SELECT STATEMENT		
TABLE ACCESS	SALES	BY INDEX ROWID BATCHED
INDEX	SALES_DATE	RANGE SCAN
Access Predicates		
SALE_DATE=TO_DATE(' 2017-01-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss')		

Indizes 19/20
ORACLE - Index-organisierte Tabelle

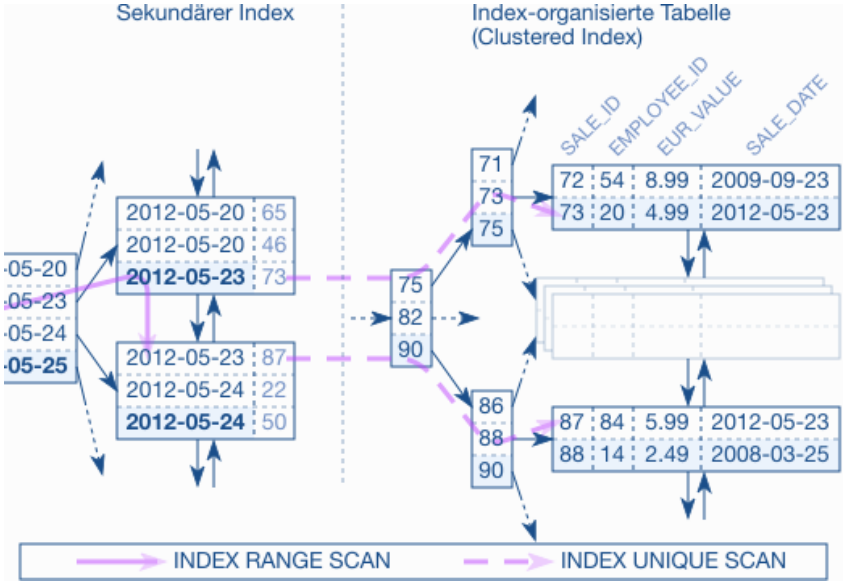
- Indexorganisierte Tabellen werden mit **ORGANIZATION INDEX**-Klausel definiert, wobei die Tupel in den Blättern gespeichert sind, geordnet nach den Werten des Primärschlüssels.

CREATE TABLE-Anweisung als IOT

```
CREATE TABLE Sales_IOT (
  sale_id          NUMBER          NOT NULL,
  employee_id      NUMBER          NOT NULL,
  eur_value        NUMBER(17,2)    NOT NULL,
  sale_date        DATE            NOT NULL,
  CONSTRAINT sales_iot_pk PRIMARY KEY (sale_id)
)
ORGANIZATION INDEX;

CREATE INDEX sales_iot_date
ON Sales_IOT (sale_date);

SELECT sale_id
FROM Sales_IOT
WHERE sale_date =
      to_date('2012-05-23', 'yyyy-mm-dd');
```



OPERATION	OBJECT_NAME	OPTIONS
SELECT STATEMENT		
INDEX	SALES_IOT_DATE	RANGE SCAN
Access Predicates		
SALE_DATE=TO_DATE(' 2012-05-23 00:00:00', 'syyyy-mm-dd hh24:mi:ss')		

Achtung: Der Zugriff auf eine Index-organisierte Tabelle über einen sekundären Index ist sehr ineffizient!

Indizes 20/20
ORACLE - Indexverwaltung

Indexverwaltung

```
DROP INDEX Indexname;
DROP INDEX Indexname RENAME TO NeuerIndexname;

ALTER INDEX Indexname UNUSABLE;
ALTER INDEX Indexname REBUILD [ONLINE];

ALTER INDEX Indexname MONITORING USAGE;
ALTER INDEX Indexname NOMONITORING USAGE;

...
```

Indexnutzung

```
SELECT index_name, table_name, used, start_monitoring, ...
FROM v$object_usage;
```

Index
Wann soll ein Index verwendet werden?

- **Kandidaten** für Indizierung
  - Primärschlüssel und `UNIQUE`-Spalten (automatisch!)
  - Fremdschlüssel-Spalten (meistens, vor allem bei Verbund!)
  - Spalten, die häufig in `WHERE`-Klauseln von signifikanten, selektiven Anweisungen verwendet werden
  - Spalten, die in der `ORDER BY`-Klausel oder von Funktionen wie `MAX` oder `MIN` verwendet werden
  - Spalten enthalten großen Wertebereich
  - Tabelle wird selten aktualisiert

- **Vorsicht bei**
  - Hoher `INSERT`/`UPDATE`/`DELETE`-Frequenz auf indizierte Attribute, da Index **reorganisiert** werden muss
  - Basistabellen mit wenigen Datensätzen (Index ab ~ 300 Zeilen)

© 2026

Index

Wann wird ein Index vom Anfrageoptimierer verwendet?

- **Kardinalität**
  = Anzahl der Werte einer Spalte mit unterschiedlichen Ausprägungen
- **Selektivität**
  = Verhältnis Kardinalität einer indizierten Spalte zur Anzahl der Datensätze

CUSTOMER

Customer_ID	Business_Size
1	S
2	L
3	M
4	M
5	S

Indexerstellung

```
CREATE INDEX Customer_Business_Size
ON Customer(Business_Size);
```

Kardinalität = 3 (S, M, L)
Selektivität = 0.6 (3/5) [bei 10.000 Customer = 0.0003]

Berechnung Selektivität in Oracle

```
SELECT (distinct_keys / num_rows) AS Selectivity
FROM dba_indexes
WHERE index_name LIKE UPPER('Customer_Business_Size');
```

Ergebnis

SELECTIVITY
0.6

Berechnung Selektivität in Oracle – Index Primary Key

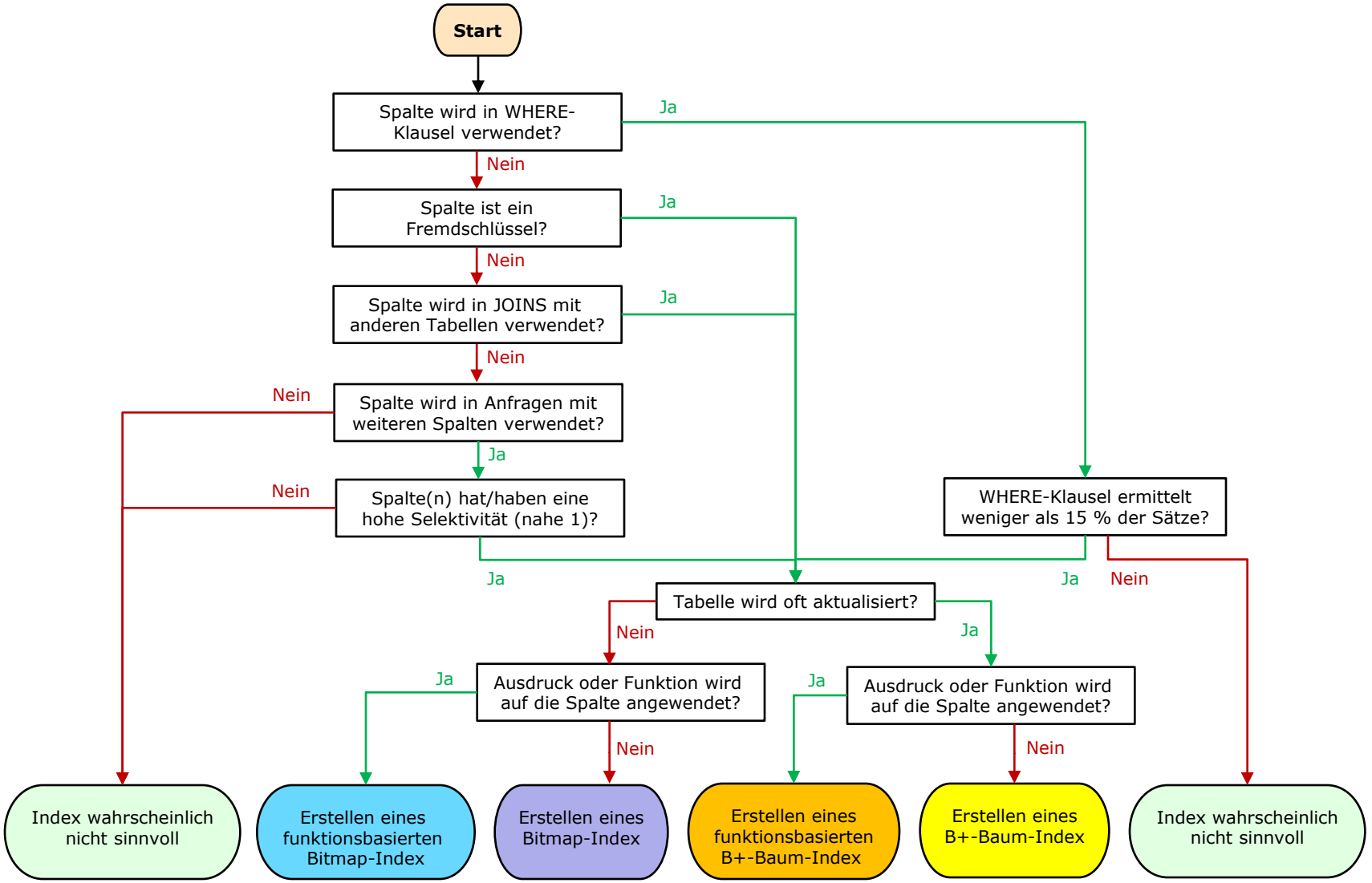
```
SELECT (distinct_keys / num_rows) AS Selectivity
FROM dba_indexes
WHERE index_name LIKE UPPER('SYS_C0017971');
```

Ergebnis

SELECTIVITY
1.0

Faustregel: Index nur bei hoher Selektivität, d.h. bei kleiner Treffermenge

Index
Entscheidungsbaum



© 2026

Zusammenfassung

- Indizes beschleunigen Punkt-, Bereichs- und Verbundanfragen erheblich.
- Indizes verlangsamen das Einfügen, Löschen und Ändern von Tupeln der Relation.
  - Indizes müssen automatisch nachgeführt werden.
- Indizes benötigen Speicherplatz und aktuelle Statistiken
- Indexerzeugung kann lange dauern.
  - Bulk Load: Es ist (viel) effizienter, zuerst die Daten in die Tabelle einzufügen und nachher alle Indizes zu erstellen als umgekehrt.

- Indizesdefinition ist eine der schwierigsten Aufgaben des Datenbankdesigns
  - Vorhersage der Query Workload und Änderungsfrequenz
  - Attributwahl
    - häufiger Vergleich mit Konstanten
    - häufiges Verbundprädikat